

Token Budget Lab – Cut the Bill

Core lab ~30 min · Stretch (At Scale) ~20 min, optional · TaskFlow, one repo, zero extra setup

Shipping fast is the cheap part; running it is where the money goes. In AI-assisted work, OpEx is now a token bill, and context is a line item, not a technical detail. This lab measures three real levers on the same small repo used all day, then a stretch section grows that repo for real so the same levers show up at a scale that isn't a rounding error.

AT A GLANCE

Slot: Day 4, Part 4 (Economies of AI Development), slides 52–65. Core lab first; stretch after if time allows.

Codebase: TaskFlow — the same small Express API used all day. No second repo, no new setup.

What it proves: the same lever, measured twice: **~60–70% fewer tokens** on TaskFlow as-is, and a bigger, self-measured number once you grow the repo in the stretch.

Companion: ai-cost-for-engineers.vercel.app — M1 How AI Bills You, M2 The Context Trap, M4 Prompt Caching, M6 Model Routing.

ON THIS PAGE

- 01 The three levers
- 02 Pre-flight
- 03 Core lab — four steps (~30 min)
- 04 Glimpse — model routing (~2 min)
- 05 Stretch — token budget at scale (~20 min, optional)
- 06 Honest caveats
- 07 Troubleshooting
- 08 Learner exercises

01 The Three Levers

- **A tight payload** — CLAUDE.md plus only the files the task touches, instead of dumping the repo.
- **Prompt caching** — for the stable parts that don't change between turns.

- **Model routing** — heavy models for the hard thinking, cheap models for tests, review, and CI (a glimpse in Section 04).

This lab asks what agentic work actually costs, and what levers reduce it, using zero new setup. No new repo, no custom pipeline, no bespoke tooling — just `/context` and `/cost`, pointed at TaskFlow, read out loud.

02 Pre-flight

- Fresh Claude Code session from the TaskFlow repo root, or continuing from Demos 1–2 — either works.
- `npm test` green before you start.
- Second tab open on ai-cost-for-engineers.vercel.app (optional, for the glimpse in Section 04).

03 Core Lab — Four Steps (~30 min)

Step 1 — Measure the naive way

Point the agent at the whole TaskFlow repo and read the context meter:

PROMPT

Read every file in `src/` and `tests/`, then tell me whether the `PATCH /api/tasks/:id` route validates its request body correctly.

RUN

`/context`

Say: "That's the default habit — 'give the agent the folder so it has context.' Note the number on screen. Everything in `src/services`, `src/store`, every test file — none of it touches body validation, and we paid to load all of it anyway."

Step 2 — Measure the tight way

Fresh turn (or `/clear` for a clean comparison), give it `CLAUDE.md` plus the two files this specific task actually touches:

PROMPT

Using `CLAUDE.md`'s architecture note, check `src/routes/tasks.js` and `src/services/taskService.js` for request body validation on the `PATCH` handler.

```
RUN
/context
```

Say: "Same question, same correct answer either way. On this repo, CLAUDE.md plus the two files this task touches is about 60% smaller than the whole-repo dump – measured on the actual files here: 10.6KB for all of src/ and tests/ vs. 4.2KB for CLAUDE.md plus the two relevant files. Read the real number off your own screen – the ratio is the lesson, not a specific percentage."

HONEST NOTE

The ~60–70% figure is verified for this repo, this exact file selection (CLAUDE.md + tasks.js + taskService.js). If you pick a different file set the number moves – that's expected, not a bug in the lab.

Step 3 – Run a short multi-turn edit both ways, compare /cost

Pick one path and actually make the fix (the PATCH truthiness bug from Demo 2 – if req.body[field] → an explicit in / !== undefined check), across 2–3 turns: ask for the fix, ask for a regression test, ask to run the suite. Do this once naive, once tight (two sessions or a /clear between them), then:

```
RUN
/cost
```

Say: "Same multi-turn edit, same outcome. With the tight context, CLAUDE.md is the same stable prefix on every turn – that's exactly what prompt caching is for. First read costs 1.25×; every read after that, within the cache window, costs 0.10×. Compounded with the ~60% smaller starting payload, the tight path's session bill drops well below the naive one. The gap between the two /cost numbers on your screen is the real number, not a slide."

Step 4 – The scale line to keep

SAY THIS, LANDING IT

"60–70% on a toy repo like this one is a rounding error – a few thousand tokens, cents either way. The same 60–70% on a 100,000-token monorepo is 60,000–70,000 tokens saved every single turn. The mechanism you just watched work on TaskFlow is the same mechanism that turns a five-figure

monthly bill into a four-figure one at your actual scale. Nothing about the ratio changes with size — only the number of zeros on the check.”

04 Glimpse — Model Routing (~2 min, optional)

The other half of the bill is which model runs the request. No new setup — Demo 1 already put this in the repo:

PROMPT

Use the doc-writer subagent to document src/routes/tasks.js.

RUN

/cost

PROMPT

Use the code-reviewer subagent to review src/routes/tasks.js.

RUN

/cost

Say: "doc-writer runs on Haiku — documentation is deterministic, routine work. code-reviewer runs on this session's own tier, because judging correctness and security is the hard thinking. Frontier models for requirements, architecture, first implementation; small, fast, cheap models for tests, review, and CI/CD monitoring. Same output quality on the routine work, a fraction of the token cost — and it's one line of YAML, already sitting in this repo."

05 Stretch — Token Budget at Scale (~20 min, optional)

TaskFlow is deliberately tiny so the mechanism above is easy to see. The honest tradeoff: 60–70% of a 10.6KB repo is a rounding error in real dollars. This stretch makes the same lever behave like it would on a repo your team actually ships — by growing the repo for real, fanning work out with a workflow, and pricing the gap — without leaving TaskFlow or standing up a separate large codebase.

Stretch Step 1 — Start from the real, shared baseline (2 min)

Clone the pushed repo cold instead of reusing your locally-modified copy — treat it the way you'd treat a teammate's repo you're opening for the first time:

RUN

```
git clone https://github.com/nsr-19/acceler-lvt-labs.git
cd acceler-lvt-labs/lvt-day4-code-demo && npm install && npm test
```

Say: "Everything after this point works the same on your local copy — this clone just guarantees we're all measuring the same starting point, which matters once we start comparing numbers across the room."

Stretch Step 2 — Grow the repo, then feel the tax (8 min)

Ask Claude Code to scaffold two more resources that mirror the existing pattern exactly:

PROMPT

Add two new resources to this API, following the exact pattern in `src/routes/tasks.js`, `src/services/taskService.js`, and `tests/tasks.test.js`: a "projects" resource and a "notifications" resource. Each needs a route file, a service file, and a test file with the same coverage shape as `tasks.js` has. Do not touch the existing tasks or users code.

Run `npm test` — confirm the suite is still green. Then repeat the naive-vs-tight measurement on a question that spans the new code:

PROMPT

Read every file in `src/` and `tests/`, then tell me whether the new notifications resource validates its request body the same way tasks does.

RUN

`/context`

Fresh turn or `/clear`, then tight:

PROMPT

Using CLAUDE.md's architecture note, check `src/routes/notifications.js` and `src/services/notificationService.js` for request body validation.

RUN

/context

Checkpoint: the naive number should have grown noticeably — there are now more files under `src/` to dump. The tight number should have barely moved — it's still just `CLAUDE.md` plus two files. So the percentage reduction gets bigger, not smaller, as the repo grows.

Say: "That's the at-scale lesson, and you just watched it happen on a repo you grew by hand instead of taking my word for a bigger checkout. The naive path pays for every file that exists. The tight path pays for every file the task touches. Those are different curves — one grows with the repo, one doesn't — and the gap between them is exactly what you just measured."

HONEST CAVEAT

Don't repeat the original ~60% figure after this step — say the number you actually measured on the grown repo instead. If it didn't move much, that's worth naming honestly too (two extra resources on top of three existing ones is still a small repo in absolute terms).

Stretch Step 3 — Multiply it with a workflow, for real numbers (6 min)

This is Build 3's fan-out, pointed at the cost question instead of a code question:

PROMPT

use a workflow to audit every route file under `src/routes/` for missing input validation. Run two passes: in the first, give each auditor the whole `src/` folder as context; in the second, give each auditor only `CLAUDE.md` plus its one route file. Report the total token cost of each pass and the per-agent average.

Watch `/workflows`, drill into a phase, and read the token totals per agent.

Say: "Step 4 of the core lab did this multiplication as arithmetic on a slide — N agents times naive payload versus N agents times tight payload. This is the same multiplication, except every number on screen came from an actual run, not a projection. A workflow auditing five route files is nothing. A workflow auditing five hundred is the same ratio, paid five hundred times."

Stretch Step 4 — Price it out for the room (4 min)

Take your own measured `/cost` numbers — either the single-turn delta from the core lab or the grown-repo delta from Stretch Step 2 — and do the arithmetic live:

- **Per-question cost, naive:** naive input tokens × your model's input price.
- **Per-question cost, tight:** tight input tokens × input price for the first turn, then × (input price × 0.1) for every turn after — the cache-read discount from Step 3.
- **Multiply by a realistic cadence:** e.g. 15 engineers × 8 similar questions a day × 20 workdays a month — multiply both totals out.

SAY THIS, LANDING IT

“Check whatever pricing page is current when you run this — the exact per-token numbers move over time, and reading a stale number off a slide is worse than pointing at the real one. What doesn't move is the shape of the math: naive cost scales with repo size times question volume; tight cost scales with task size times question volume. Same lever as ten minutes ago — now with a number the room can put in a budget doc instead of a percentage on a slide.”

06 Honest Caveats

- **The core-lab figure** (~60–70%) is verified for TaskFlow at its current size, this exact file selection. It moves as the repo (or the file selection) changes — that's expected, not a bug.
- **The stretch's grown-repo figure** is whatever you actually measured after Stretch Step 2 — don't default back to the original 60% once the repo has changed shape.
- **/cost's cache breakdown** varies by Claude Code version — some surface `cache_read_input_tokens` directly, some show one total. Either way, the gap between two `/cost` readings is the real number.
- **Pricing numbers** in Stretch Step 4 move over time — pull the current rate from whatever pricing page is live when you run this, not from a cached figure in this guide.

Wrap-up line

SAY THIS, LANDING IT

“Tight payload, caching, model routing — three levers, all measured on the same repo, in the same lab, with tools already in your hand. Nothing here needed a bigger repo or a custom tool. The bill was always visible; `/context` and `/cost` were just sitting there the whole time. And if a toy repo made the ratio feel small — the stretch just showed you the same ratio holds as the repo grows; only the number of zeros on the check changes.”

07 Troubleshooting

- **Step 2's reduction looks smaller/bigger than ~60%** — Expected — TaskFlow is tiny and session overhead is a bigger share of a small number. The ratio is the lesson, not the exact percentage.
- **/cost doesn't break out cache tokens** — Expected on some Claude Code versions — read the two totals and the gap between them instead.
- **Room asks why TaskFlow doesn't just always use Haiku** — Routing is per-task, not per-repo — code-reviewer needs the reasoning tier precisely because judgment calls are the hard part Haiku is bad at.
- **Stretch Step 2's grown repo doesn't npm test green** — Read the failure — usually a missed `beforeEach(() => store.clear())` or a copied route param. Fix before measuring; a red suite makes the token numbers a distraction.
- **Stretch Step 2's percentage barely moved** — Two extra resources on a three-resource API is still small in absolute terms — say so honestly, and ask what a tenfold growth would do instead of forcing the live number.
- **Stretch Step 4's price feels too small to matter** — That's the point of the multiply-out — a fraction of a cent per question looks irrelevant until multiplied by a team's real question volume.

08 Learner Exercises (self-paced)

- Re-run Steps 1–2 on a task you'd actually ask in your own repos, with your own file selection, and compare /context. Does the ratio hold?
- Open ai-cost-for-engineers.vercel.app → M4 (Prompt Caching) for the write/read multiplier table, and M6 (Model Routing) for the general version of the doc-writer/code-reviewer split.
- Add a fourth subagent to this repo with model: haiku for a task you think is over-provisioned today.
- Repeat Stretch Step 2, but grow the repo by five resources instead of two. Plot naive vs. tight /context against resource count — does the curve match the “tight cost doesn't scale with repo size” claim?
- Rerun Stretch Step 3's workflow with a deliberately bad naive prompt (e.g. include `node_modules`) to find the ceiling on how bad naive can get before someone notices the bill.
- Take the Stretch Step 4 arithmetic and run it with your own team's real headcount and question volume. Is the monthly gap worth a sentence in a planning doc?